

tcb/box tcb/boxSect
 tcb/box/begin0 tcb/box/begindefault
 tag=tcb/box tcb/box/begindefault
 tcb/box/end0 tcb/box/enddefault
 tcb/box/enddefault
 tcb/box/title tcb/box/titleCaption
 tcb/box/title0 tcb/box/titledefault para/semantictcb/box/title tcb/box/titledefault
 tcb/box/draw2 tcb/box/drawdefault 2
 tcb/box/drawdefault
 tcb/box/init0 tcb/box/initdefault minipage/beforenoopminipage/afternoop tcb/box/init-
 default
 tcb/box/upper0 tcb/box/upperdefault minipage/beforetag/dflminipage/aftertag/dflt tcb/box/up-
 perdefault
 tcb/box/lower0 tcb/box/lowerdefault minipage/beforetag/dflminipage/aftertag/dflt tcb/box/low-
 erdefault
 tcb/drawing/init0tcb/drawing/initdefaulttcb/drawing/initdefault

Boon Academy — Intervention Triage

A facilitator-mediated, closed-loop system for getting help to failing students before the next quiz — technical report

AI Systems Engineer case study

June 2026

Boon Academy already knows *who* is failing; only ~30% of those students get any intervention before the next quiz. The bottleneck is not detection — it is **triage, action, and never knowing whether the help worked**. This system produces a ranked, capacity-bounded *action queue* per facilitator by fusing a transparent rule engine (the numbers) with an LLM that *reads* the Arabic facilitator notes (the story), and then — the part that distinguishes it — **measures whether past interventions actually re-engaged students**. On the provided 200-student sample, only **11%** of logged interventions show re-engagement: a logged note is not an effective intervention, and that gap is the real problem behind the 30%. Every design choice is grounded in the early-warning and behavioural-nudge evidence base and stated with its limitations.

Contents

1	The problem and the diagnosis	2
2	The data	3
2.1	The data traps (handled once, in ingest)	3
2.2	Data insights	3
2.3	How facilitators differ in note-taking	4
2.4	The demo cast — where numbers and notes disagree	5
3	Architecture	5
4	The risk engine (NEED) — where a simple rule does the job	6
5	The note-reader — the only place AI is used, and why	6
5.1	Model choice	7
5.2	The prompt	7
6	The decision levels (fusion)	8

7	v2: closing the loop — measuring whether help worked	8
7.1	Fairness	9
8	The evidence base	9
9	Results and how quality is measured	10
10	Limitations and what I would build next	10

1 The problem and the diagnosis

The brief frames a fictional “Boon Academy”: students attend in person, on-site **facilitators** run each classroom, and teachers instruct remotely. When a student falls behind the facilitator is supposed to intervene (call the parent, 1-on-1 tutoring, a motivational message), but only ~30% of failing students are reached before the next quiz, and the org is scaling 1,000 → 5,000 students.

The real problem is not an AI problem

A facilitator running a full classroom cannot act on a wall of red flags — it is **cognitive overload and decision fatigue**. If she can make three calls today, the system must tell her *which three, why, and what to say*. Three deeper truths shape the design: (1) the signal is split between LMS *numbers* and the facilitators’ own free-text *notes*, and nobody synthesises them; (2) attendance is a deceptive proxy; (3) a logged note ≠ an effective intervention.

2 The data

Three CSVs joined on `student_id` only (names written inside notes do not match the roster). 200 students across 5 campuses and 8 facilitators; 10 active class-days (1–14 Oct 2025, weekends absent); Quiz 1 on day 10; the system’s “today” is a parameter (`AS_OF_DATE=2025-10-14`), never `now()`, so runs are reproducible.

```
metadata student_id, student_name, campus_id, facilitator_email, grade, parent_phone,
  target_score, learning_track
daily_metrics student_id, date, session_attended_min, practice_questions, last_quiz_score,
  days_until_next_quiz
notes note_id, student_id, facilitator_email, date, note_text (180 notes, 100%
  Arabic free text)
```

2.1 The data traps (handled once, in ingest)

Real operational data is messy; each trap is neutralised in one place so the rest of the pipeline can trust the record.

- **Name mismatch** — join on `student_id` only.
- **Carried-forward quiz** — `last_quiz_score` repeats across days; it is one event (taken at the quiz date), not a daily score.
- **Quiz 0 that is an absence** — a 0 with a *blank* session on quiz day means “absent during the quiz”, not a real failing score (student S145).
- **Corrupt target_score** — a whole Remedial cohort has targets below the pass mark (e.g. 12); never used as a signal.
- **Dirty phones** — a missing +, an email in the phone field — normalised and flagged, never echoed raw (privacy).
- **Blank cells** — kept distinct from a real 0.0.

2.2 Data insights

Table 1: Quiz-1 failure by learning track — track is nearly destiny.

Track	Failed / total	Fail rate
Accelerated	0 / 14	0%
Standard	15 / 128	12%
Remedial	52 / 58	90%
All	67 / 200	34%

- **Coverage & blind spots:** only **75/200 (38%)** students have any note; **125 (62%)** were never contacted, including **20 who failed the quiz and have no note at all** — invisible to any notes-based view.
- **Attendance is deceptive:** ~15 students attend ~88 min every day, do almost no practice, and score in the 50s–low-60s. An attendance-only triage rates them flawless; only practice + quiz catch them. Hence practice is a *separate* axis from attendance.
- **More practice can mean more risk:** one student did 120 practice questions in a single day (cramming) and still failed — so raw volume is not rewarded; the engine uses the spike-robust *median* and flags single-day spikes.
- **Notes are reactive:** 74 of 180 notes (41%) land on the single final day, with secondary spikes on days 3–4 (enrollment) and day 13. Notes are written *after* the fact, not as early warning.

2.3 How facilitators differ in note-taking

Note-taking is far from uniform, and the differences map onto the overload story directly.

The overload is visible in the data

The two facilitators with the *largest* rosters (38 and 35 students) have the two *lowest* note-coverage rates (21% and 34%) and, for the 38-student case, the *tersest* notes (median 9 words). Bigger caseload → less documentation → more students falling through. This is precisely the capacity problem the system is built to relieve — and a reason the action queue is **capacity-bounded per facilitator**, not a global dump.

Table 2: Per-facilitator note-taking. Notes are short (median 12 words, range 2–23) and uniformly Arabic; coverage and verbosity vary widely.

Facilitator	Roster	Noted	Coverage	Notes	Median words
facilitator1	20	9	45%	27	13
facilitator2	20	8	40%	19	14
facilitator3	35	12	34%	26	14
facilitator4	22	11	50%	24	10
facilitator5	20	9	45%	23	12
facilitator6	24	9	38%	19	10
facilitator7	21	9	43%	21	11
facilitator8	38	8	21%	21	9

Practical consequences for the LLM: notes are *short and dialectal* (12 words median), so the reader must extract a lot from little and tolerate Najdi/Hijazi spelling, code-switching, and abbreviations; and because *count/recency* of notes varies by facilitator habit rather than by student need, note metadata is a poor risk signal — another reason to read *content*, not count.

2.4 The demo cast — where numbers and notes disagree

The dataset is built around students whose numbers and notes point in opposite directions; these drive the demo and the acceptance tests.

Table 3: Representative students. The point of reading notes is the right-hand column.

ID	Numbers say	Notes say	Correct call
S023	passing (88)	vanished, no reply	Critical (silent dropout)
S005	passing (85)	“worried, needs immediate follow-up”	Critical (failing intervention)
S145	quiz 0	“big transformation, dad engaged”	Low (the 0 is an absence)
S017	passing (83)	“crying — family problems”	Surface (note-only crisis)
S199	quiz 9, target 12	(no note)	Critical (invisible at-risk)
S051	quiz 52	“practising regularly” (120-q cram)	High (do not demote)
S076	passing (84)	“excellent student”	Leave alone

3 Architecture

A deliberately small linear pipeline (~1,450 lines of Python; pandas + the Google **genai** SDK) with **one** LLM seam. Four objects flow through it:

StudentRecord → Risk + NoteState → ActionRow → queue.

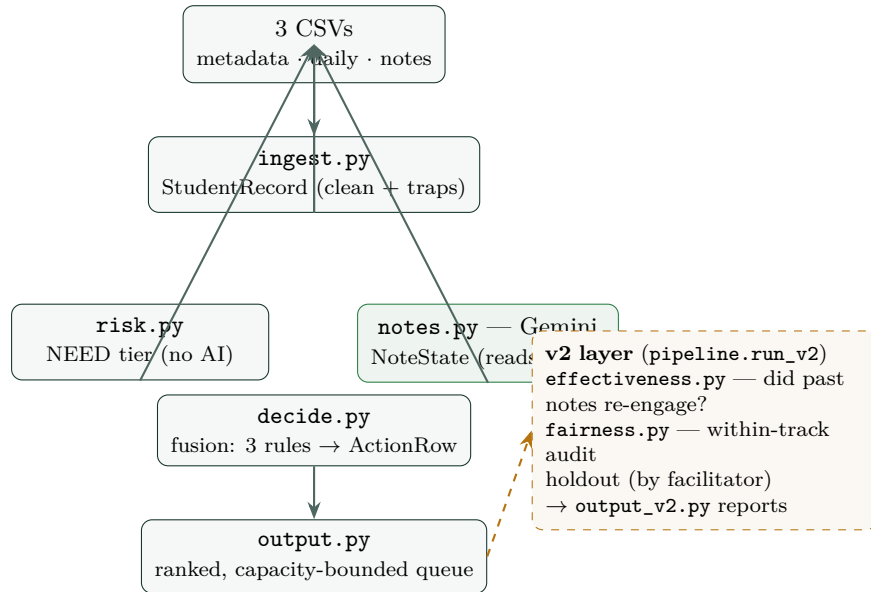


Figure 1: Data flow. `config.py` feeds constants everywhere; `pipeline.py` wires it; `main.py` is the CLI. The only LLM call is the green box.

4 The risk engine (NEED) — where a simple rule does the job

Deterministic and explainable on purpose: 14 days of data, no labelled outcomes, and a decision about children all argue for transparent rules over a black box. Four contributors, severity-ordered, each attaching a human-readable reason:

Coursework failed Quiz 1 (absence-flagged scores excluded).

Attendance chronically low recent attendance (only when it is *not* a cliff, to avoid double-counting).

Behaviour little *sustained* practice (median, so a cram spike is not credited).

Trajectory a strong attendant who *collapses* to near-zero in the last days — a silent dropout the quiz can never catch (S023).

Points map to a tier (Critical/High/Medium/Low). Notably, `target_score` is *excluded* (corrupt for a whole cohort) and `learning_track` is *not* an input (fairness, §7.1).

5 The note-reader — the only place AI is used, and why

The one job a rule cannot do is read short, informal, dialectal Arabic prose. So the LLM is pointed at exactly one thing — the notes — for two tasks a rule can't: **reading** them into a structured state that changes the priority, and **drafting** the parent message. It never scores risk.

Table 4: Files and responsibilities (~1,450 lines total).

File	Job	Lines
<code>config.py</code>	tunables, <code>.env</code> loader, <code>AS_OF_DATE</code>	66
<code>ingest.py</code>	3 CSVs $\rightarrow$ StudentRecord; every trap, once	212
<code>risk.py</code>	NEED: 3 signals + cliff $\rightarrow$ tier + reasons (no AI)	79
<code>notes.py</code>	the LLM seam: Gemini reads notes $\rightarrow$ NoteState	231
<code>decide.py</code>	fusion: NEED $\times$ STATE, 3 rules $\rightarrow$ ActionRow	148
<code>output.py</code>	rank, capacity-bound, write CSV/JSON/HTML	173
<code>pipeline.py</code>	orchestrator: run / run_v2 / compute_lift	119
<code>effectiveness.py</code>	v2: retrospective “did it work?”	124
<code>fairness.py</code>	v2: within-track flag audit	48
<code>output_v2.py</code>	v2: write the reports	66
<code>tests/test_cast.py</code>	13 acceptance tests (the demo cast)	109

Extractor

The LLM emits a small structured *state*; deterministic rules decide. This bounds the blast radius of any wrong read: it can escalate, demote, or flag-for-review, but it can never silently drop a failing student (the numbers floor always surfaces them), and a working-note can never demote a quiz-failing student (a guarded rule). The model proposes; the rules dispose.

5.1 Model choice

Gemini 2.5 Flash Lite, temperature 0, via the API behind a swappable `LLMBackend` interface (a `NoLLM` backend gives the rules-only baseline).

Why this model

Native structured output (`responseSchema`) guarantees schema-valid JSON — which kills the single biggest execution risk (a model emitting broken JSON from dialectal Arabic). It is cheap (~\$0.0003/student) and fast. Because it is one isolated module, production with real minors’ PII under PDPL swaps it for a KSA-region or on-prem Arabic model *without touching the pipeline*. A second finding shaped the message design: current models *read* Gulf dialect well but *generate* it poorly — so messages are drafted in warm, simple Modern Standard Arabic (the natural polite register for a Saudi family message anyway), not synthetic colloquial.

5.2 The prompt

The system prompt (lightly abridged) — the schema *is* the architecture:

```
You read a single facilitator's notes about ONE student (short, informal Saudi-dialect
Arabic).
Your ONLY job is to report the STATE of any intervention -- you do not score risk. Return
JSON:
- state: on_track | needs_help | failing | refused | none
```

on_track was struggling, now improving / intervention clearly working
 failing intervention tried but NOT working -- parent/student unresponsive, worsening
 refused parent or student declined help
 needs_help a real problem/crisis, attention needed, no working intervention yet
 none descriptive only / no clear signal

- summary one plain sentence: what is going on
- root_cause e.g. gaming, family issue, unaware of program (or "")
- suggested_action call_parent | one_on_one | message | none
- draft_message a short, warm WhatsApp message in SIMPLE Modern Standard Arabic, to the PARENT,
 sent BY the named facilitator: cite ONE specific fact; be improvement-focused (not praise/blame);
 propose exactly ONE doable step (NEVER a meeting); 2-3 sentences.
- evidence an EXACT short Arabic span copied from the notes
- concern low | neutral | worried | urgent
- confidence low | medium | high (use low when vague -- do not guess)

Be faithful. Do not invent facts.

Why every field earns its place

- **state** drives the fusion override — it is the thing a rule cannot derive.
- **evidence** (a verbatim Arabic span) enables a *faithfulness check*: the quote must be a substring of a real note, or it is dropped and confidence lowered — the anti-hallucination floor.
- **confidence** powers *abstention*: low-confidence reads go to a human-review lane, never auto-acted.
- **draft_message** encodes the behavioural-nudge evidence (§8): *parent-directed, improvement-focused, one doable step, no meeting* (the Saudi working-parent barrier), sent by the named facilitator (the “trusted local sender”).

Two few-shot examples teach the dialect mapping. A failing case (تدرام، مالا بلع نيترم تلصتا) maps to state=failing, concern=urgent; a turnaround (ريبك لوت) maps to state=on_track. The drafted message stays warm MSA, e.g. مكيلع ماسلا. نكل رداق بلاط دمحا، رابتخالا لبق انسحت عقوتين مونلل تبث تقو بلع هم قفتن ول؛ عويسلا اذه عجارت هروضح.

6 The decision levels (fusion)

Three gating rules combine NEED (numbers) and STATE (notes); deliberately three, not a matrix — the whole point of reading notes is to correct the two cases where the numbers lie.

1. **Escalate** — state is *failing/refused*: push up; a recent contact must NOT lower risk if the intervention is failing (S005).
2. **Demote** — state is *on_track* **and** the numbers agree (not failing, not a cliff): lower it to free capacity (S145, S070). The “numbers agree” guard is what stops a soft note from rescuing a quiz-failing crammer (S051).
3. **Human review** — a low-confidence/vague note: surface for a human, never auto-act (S013).

Surfacing rule: a student is queued if the numbers flag them *or* the note is non-positive — the single OR that catches S017 (clean numbers, a hidden family crisis). The queue is then ranked and **capacity-bounded** to the top- N per facilitator.

7 v2: closing the loop — measuring whether help worked

A ranked queue is necessary but, on its own, insufficient: the early-warning literature shows that identifying at-risk students reliably fails to improve outcomes unless the data drives a real, completed intervention (§8). v2 adds the measurement layer.

The headline: attempted $\neq$ effective

Every facilitator note is a dated intervention; the metrics are daily. So we measure what each student's engagement did *after* contact. Of **75 logged interventions: only 11% re-engaged the student, 11% kept declining despite contact, and 13% were logged too late to act** (final-day notes). A logged note is not help. This measured gap is the real story behind the 30%.

- **Note-reader, checked against reality:** on the students whose engagement decisively moved, the LLM's read agreed with the actual outcome **100%** of the time ($n = 14$) — a data-grounded accuracy check (**working** $\rightarrow$ re-engaged; **failing** $\rightarrow$ declined).
- **Closing the loop:** 6 surfaced students were *already contacted and still declined* — they do not need another drafted message; they are flagged for **escalation beyond messaging** (S005, S023, ...).
- **Measurement arms:** an always-on holdout, randomised *by facilitator* (cluster, so parents/facilitators talking cannot leak treatment), reserves a control group so program lift can be measured rather than assumed.

7.1 Fairness

Guarding against the documented early-warning harms

The risk score uses *behaviour only* (attendance, practice, quiz, trajectory) — never **learning_track**, **campus**, name, phone, or grade. The audit reports surface-rate by track (Remedial is surfaced more because it is genuinely failing more, by behaviour, not by membership), and a *within-track* view guarantees the most-at-risk Standard/Accelerated students are still seen — and that Remedial membership never becomes a self-entrenching re-flag. This directly answers the real-world failures where recall-maxed dropout models concentrated false alarms on the most disadvantaged with no benefit.

8 The evidence base

The design is not invented; each choice traces to the early-warning and behavioural-nudge literature.

- **Prediction without action fails.** A 73-school RCT of an early-warning system improved intermediate proxies but *not* the ultimate outcome; a 38-study review found dashboards do not improve achievement. *So we built an action layer, and measure effectiveness.*
- **Personalised, parent-directed nudges causally work**, concentrated on the dropout / re-engagement margin and among the highest-risk students; improvement-focused content beats praise. *So the message is parent-directed, improvement-focused, one step.*
- **The trusted local sender is load-bearing** — nudges that worked locally collapsed to a precise null at national scale when the sender became impersonal. *So the facilitator sends from their own name; we never centralise to “Boon”.*
- **Arabic LLMs read dialect well but generate it poorly.** *So we extract with the model and draft in warm MSA.*

Confirmed effect sizes are modest (single-digit percentage points). **None** of the causal evidence is Saudi or Arabic — external validity for Boon’s context is a hypothesis to pilot, which is exactly why the holdout is built in. “Two-way beats one-way” and “nudges survive at scale” did not survive verification and are treated as open.

9 Results and how quality is measured

- **Runs end-to-end**, one command, LLM on; committed **outputs/** (queue + Arabic drafts).
- **13/13 acceptance tests** pass — the demo cast, including the two bug-fix guarantees (S051 cannot be demoted; S145 never reads as a real failure).
- **LLM lift:** the notes re-prioritised **13 of 75** noted students vs. rules-only (5 hidden crises surfaced, 8 failing/refused interventions escalated).
- **Note-reader accuracy vs. outcomes:** 100% directional agreement ($n = 14$).
- **Effectiveness:** 11% re-engaged / 11% declined / 13% logged too late, of 75.
- **Faithfulness** (evidence must be a real substring) and **abstention** (low confidence → human) guard the AI; quality is reproducible (temp 0 + content-hash cache).

10 Limitations and what I would build next

The decision logic is tuned against a 200-student sample (it is both the design target and the test set); the $n = 14$ accuracy check is small; the effectiveness “no-change” bucket includes stable passing students, so the headline percentages are directional, not a clean trial; and the holdout is a *mechanism* on a static snapshot, not yet a measured lift.

Next, gated on richer data and scale: a per-student *temporal narrative* (the shape of the decline, not a tier); *knowledge tracing* to diagnose the specific concept a student fails (needs per-question data); a *student-voice* two-way check-in; and a causal *what-works-for-whom*

loop (uplift / bandits) once volume justifies it. The single most important addition is turning the holdout into a live measurement of realised intervention lift — moving the metric from *attempted* to *effective*.